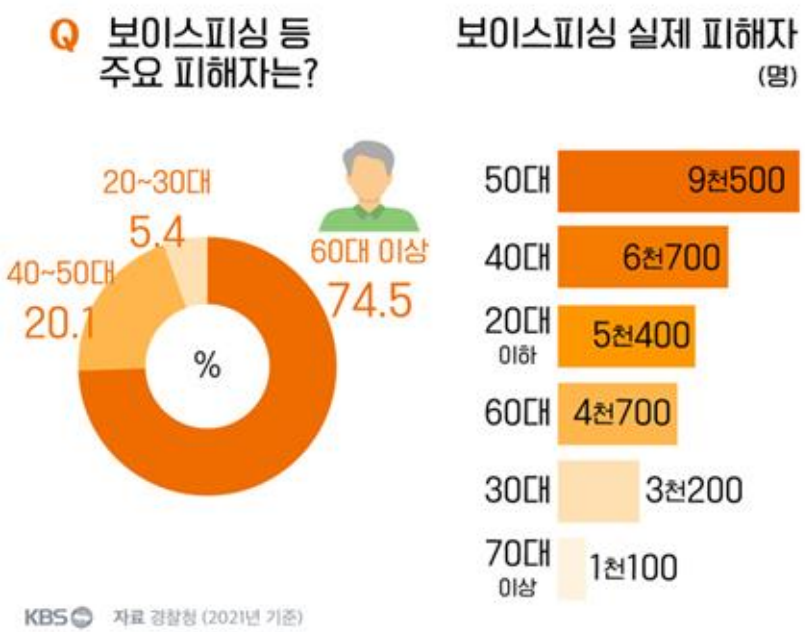
SW제작 설계서

프로젝트 명 : **AI NLP를 이용한 보이스 피싱탐지(그놈목소리)**

2025. 07. 14

팀명: 피싱제로즈

| 시장/기술 동향 분석



-보이스피싱 피해는 지속적으로 증가하고 있으며, 피해 규모 또한 심각한 수준에 도달하고 있습니다. 위 자료는 금융 서비스에 익숙하지만 최신 IT 기술에는 다소 취약한 세대를 타겟으로 한 지능적인 범죄가 활발해졌다는 점을 시사하는데, 특히 최근에는 AI 음성합성 기술을 악용한 보이스피싱 시도까지 등장하면서, 기존의 단순 필터링이나 전화번호 차단만으로는 탐지가 어려운 상황입니다.

이동통신 3사 AI 기반 보이스피싱 탐지 서비스 현황

※ 출처: 각 사

SK telecom	• 온디바이스 AI 기반 실시간 보이스피싱 탐지 기술 연내 출시 목표로 개발 중 • 사내 보이스피싱, 스팸 · 스미싱 피해예방 전담조직 운영
kt	• 온디바이스 AI 기반 실시간 보이스피싱 탐지 기술 연내 출시 목표로 개발 중 • 보이스피싱 번호를 네트워크상에서 막는 '긴급 망 차단 서비스' 제공
LG U+	• AI 통화비서 앱 '익시오(ixi-O)'에 온디바이스AI 보이스피싱 탐지 기술 탑재 • 보이스피싱으로 의심되는 번호 감지 시 '보이스피싱 위험 알림' 제공

온디바이스 AI란? 온디바이스AI는 인터넷 통신 없이도 디바이스 내에서 정보를 처리하는 AI.

보이스피싱 탐지 기술은 이미 상용 서비스 단계에 도달해 있으며, 기술의 완성도 또한 매우 높은 수준입니다.

그럼에도 불구하고 본 프로젝트는 Whisper 기반 STT(Speech-to-Text)와 KoBERT 기반 분류 모델을 결합한 LLM(Large Language Model) 응용 사례라는 점에서 의의를 가집니다.

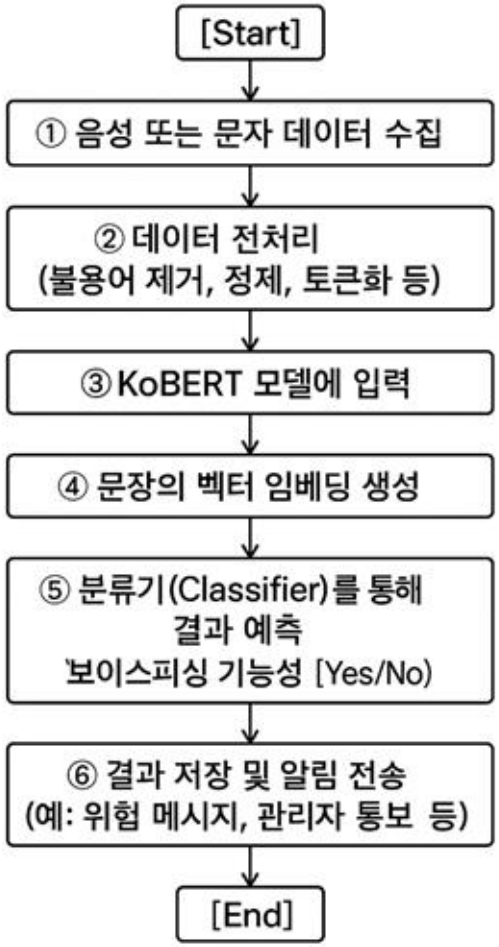
특히, 단순한 스팸 탐지를 넘어 대규모 언어 모델을 활용하여 범죄 맥락을 이해하고 탐지하는 경험을 축적한다는 점에서 차별성을 가진다. 이는 곧 향후 다양한 범죄 탐지·예방 서비스로 확장될 수 있는 실험적 기반이 될 것입니다.

| 요구사항 정의서

구분	기능	설명
S/W	STT 모듈	OpenAI Whisper Large 모델을 활용하여 업로드된 음성 파일을 문장 단위 텍스트로 변환
	텍스트 분류 모듈	KoBERT 기반 분류 모델을 통해 STT 결과를 분석하고 정상/보이스피싱 여부 판별
	백엔드 서버	Flask 서버로 파일 업로드, STT 처리, 모델 예측, JSON 형태로 결과 반환
	프론트엔드 웹	React 기반 웹에서 음성 파일 업로드, 탐지 결과 시각화 제공
	데이터 전처리 도구	STT 텍스트에서 불필요한 발화 제거, 라벨링 일관성 유지
	통신 모듈(API)	HTTP POST 요청 기반으로 파일 전송 및 결과 반환

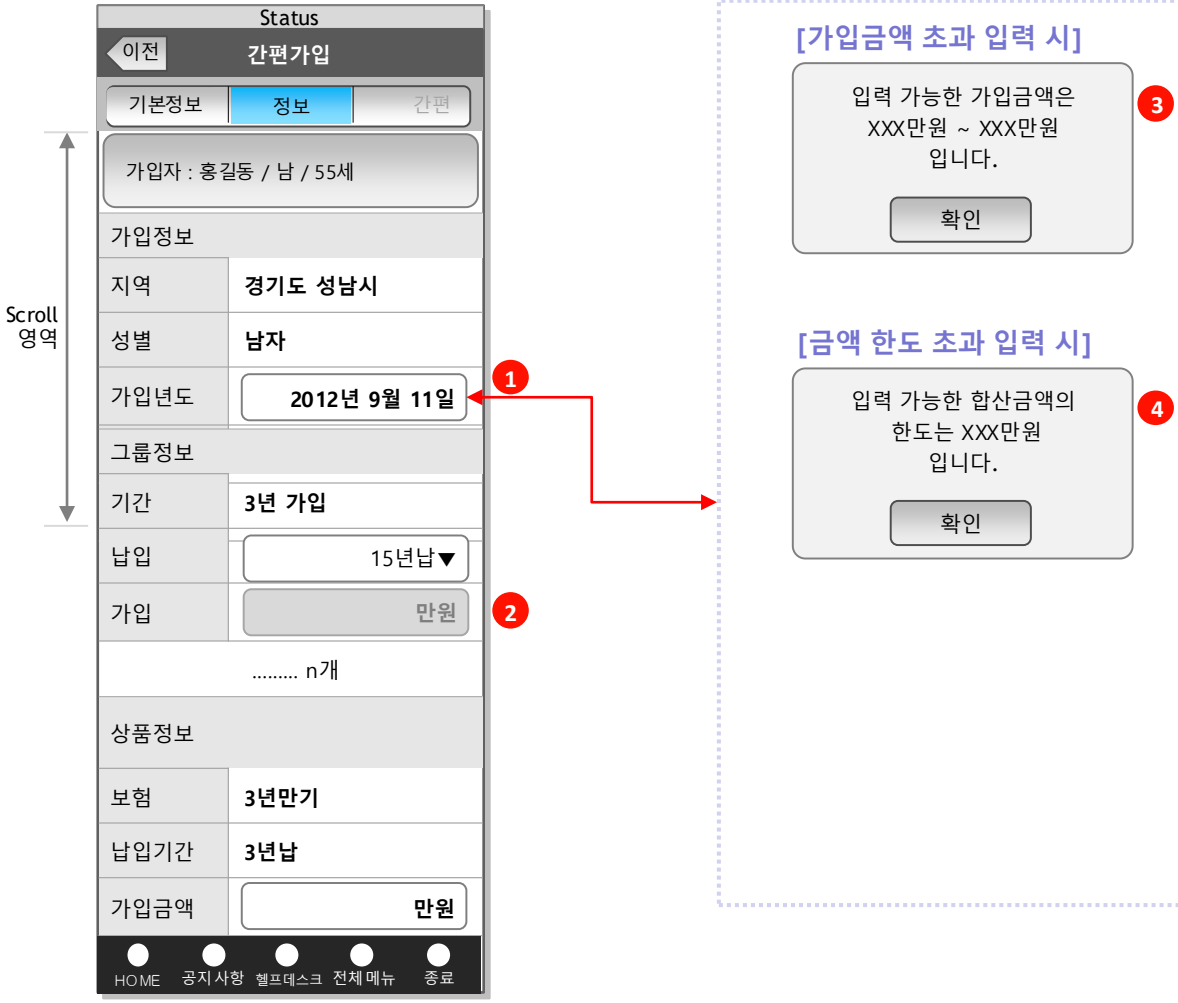
구분	기능	설명
H/W	GPU 서버	Naver Cloud Platform(NCP) GPU 서버 사용, Whisper Large + KoBERT 실행 환경 제공
	스토리지	전처리된 음성 텍스트 데이터 및 모델 가중치 저장
	클라이언트 PC	프론트엔드 웹에 접속하여 파일 업로드 및 탐지 결과 확인 가능
	네트워크 환경	외부 접속이 가능한 IP와 포트(49.xxx.xxx:5000) 기반 서비스 운영

| 서비스 구성도



1. StartPoint: 사용자로부터 수신된 음성 또는 문자 데이터를 수집.
2. 전처리 단계: 텍스트 정제, 토큰화, 특수문자 제거 등 수행.
3. KoBERT 모델 입력: 전처리된 데이터를 KoBERT 모델에 입력하여 문장의 의미를 임베딩.
4. 분류기 통과: 보이스피싱 여부를 판단하는 Softmax 분류기 사용.
5. 결과 판별: 보이스피싱 확률이 임계값(예: 0.5) 이상이면 "보이스피싱 의심"으로 표시.
6. 후처리: 탐지 결과 저장 및 사용자 또는 관리자에게 경고.

- 회원가입 신청 > 가입 상품 리스트 > 기본 정보 입력 > 추가 정보 입력

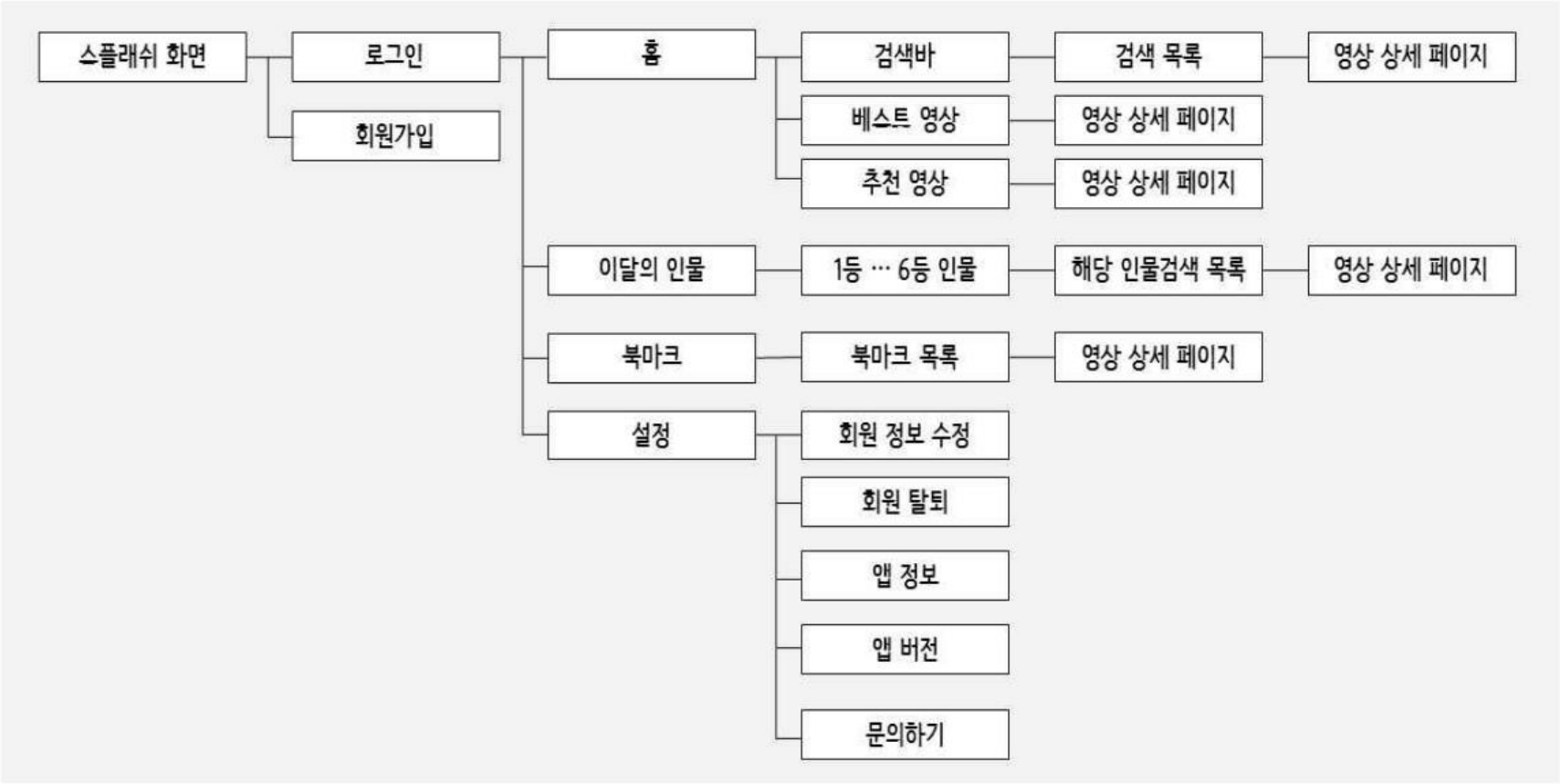


	▪ *리스트 참고*
1	**가입년도** ▪ Tap 시 숫자 Keypad 올라옴 (입력박스 가려지지 않도록 함) ▪ '만원' 으로 표기 ▪ 금액 표기 정책 참고 ▪ *가입범위 : 제공 리스트 참고*
1	**가입 (팝업 제공하는 경우)** ▪ *가입범위를 넘을 경우* ▪ *합산 한도를 초과할 경우* ▪ *가입 시 입력박스에 동시에 입력할 경우*
2	
3	
4	
5	

※ 본문의 예시 내용을 지우고 과제 내용으로 변경하여 사용하세요

| 메뉴 구성도

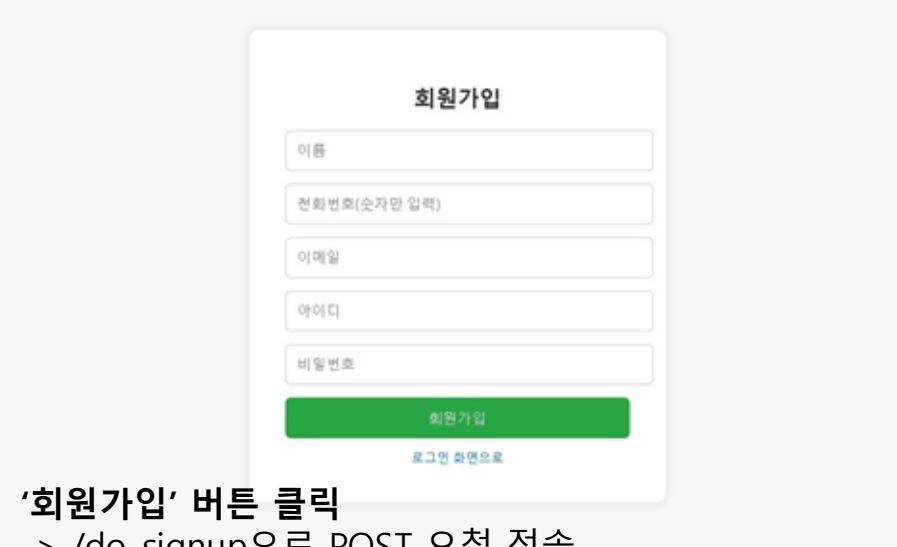
ㅇ 메뉴 구성도



※ 본문의 예시 내용을 지우고 과제 내용으로 변경하여 사용하세요

| 화면 설계서

화면 이름	회원가입
화면 목적	사용자로부터 사용자의 정보를 입력받아 계정 생성
구성요소	• 입력필드(모두 필수입력) - 이름(username) : 텍스트 - 전화번호(phone) : 숫자만 허용, 정규식 검증(10~11자리) - 이메일(email) : 형식 자동 검증 - 아이디(userid) : 고유값, 중복 불가 - 비밀번호(password) : 입력 필드 숨김 처리 • 버튼 - 회원가입 : 폼 제출 - 로그인 화면으로 : 로그인 페이지로 이동



'회원가입' 버튼 클릭
-> /do_signup으로 POST 요청 전송
-> 성공 시 : 로그인 페이지로 이동

-> 실패 시 : 로그인 화면으로 이동한 후 오류 메시지 표시

| 화면 설계서

화면 이름	로그인
화면 목적	사용자가 회원가입 시 입력한 아이디와 비밀번호로 로그인
구성요소	• 입력필드(모두 필수입력) - 아이디(userid) - 비밀번호(password) • 버튼 - 로그인 : 로그인 요청 전송 - 회원가입 : 회원가입 페이지로 이동

'로그인' 버튼 클릭
-> /do_login으로 POST 요청
-> 성공 시 : 업로드 페이지로 이동

-> 실패 시 : 오류 메시지 표시

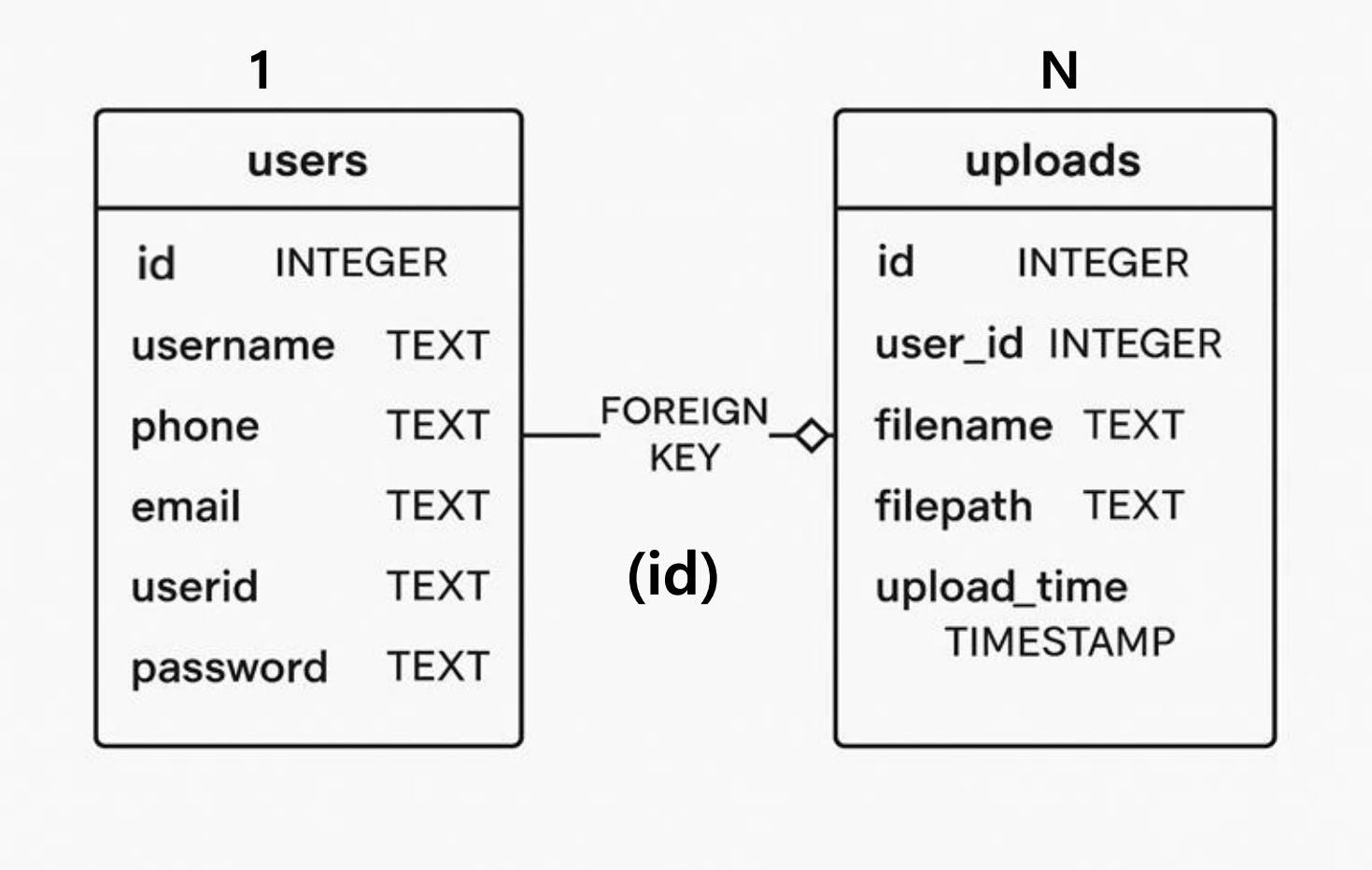
| 화면 설계서

화면 이름	음성 파일 업로드
화면 목적	로그인 한 사용자가 음성 파일을 업로드하고 서버에 저장
구성요소	• 파일 선택 필드(필수) - file : 오디오 파일만 허용 • 버튼 - 업로드 : 파일 업로드 요청 전송 - 분석 : 업로드된 음성 파일을 분석

'업로드' 버튼 클릭
-> 서버에 POST 요청
-> 파일 저장 후 DB에 기록
-> 성공 시 : 성공 메시지 표시
-> 실패 시 : 실패 메시지 표시

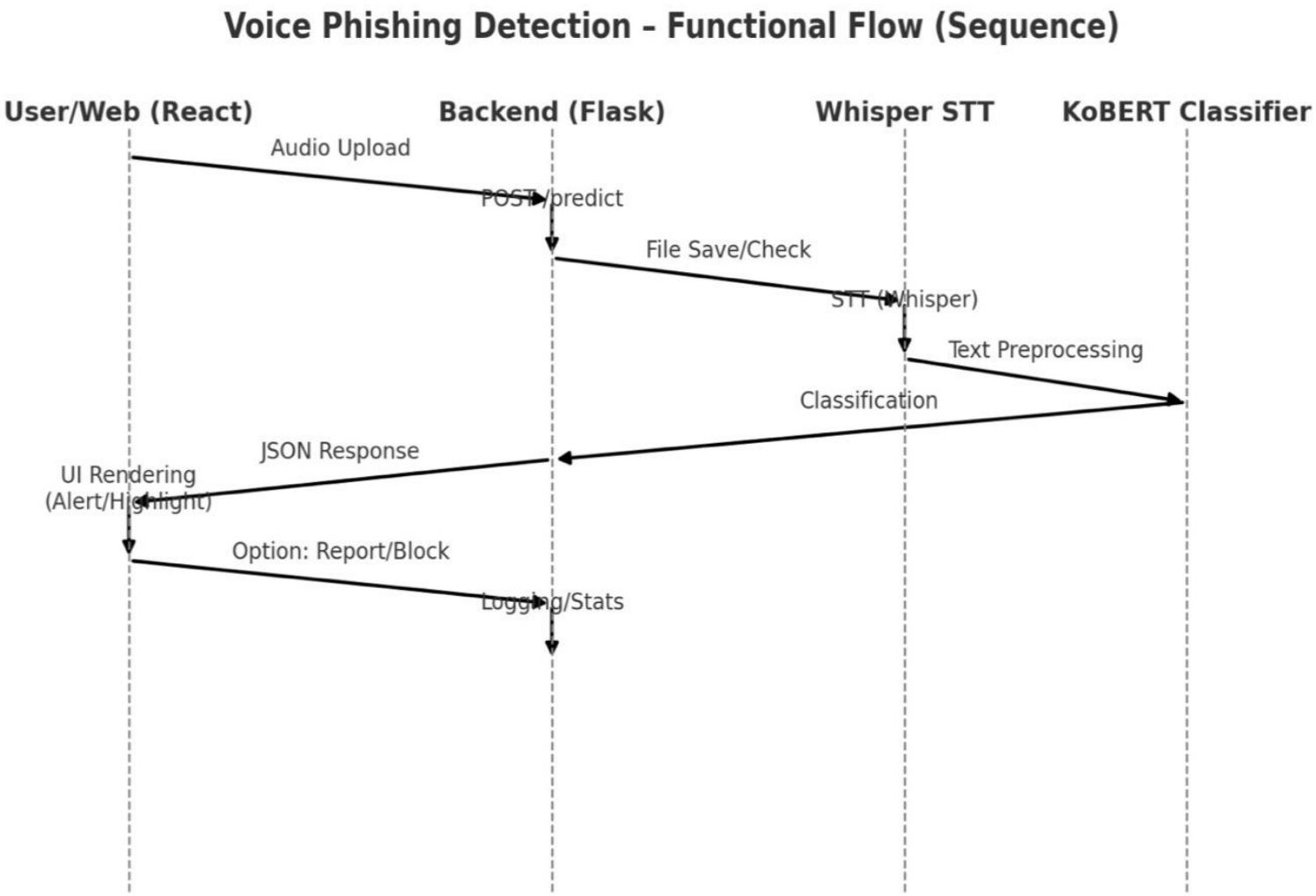
'분석' 버튼 클릭
-> 업로드된 음성 파일을 분석

| 엔티티 관계도 - ERD

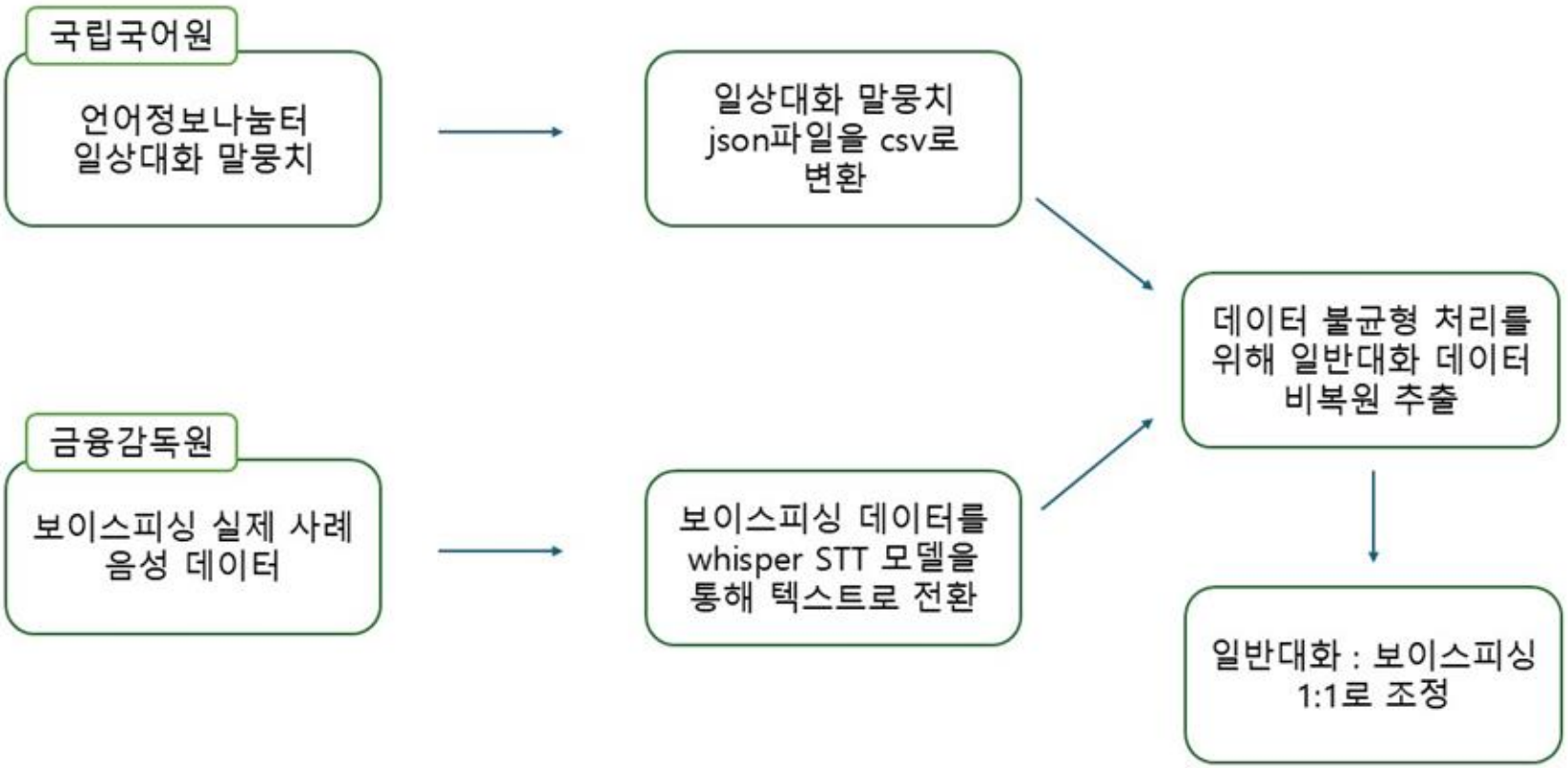


하나의 사용자는 여러 파일을 업로드할 수 있지만
하나의 파일은 하나의 사용자에만 속합니다

| 기능 처리도(기능 흐름도)


Voice Phishing Detection - Functional Flow (Sequence)
User/Web (React)
Backend (Flask)
Whisper STT
KoBERT Classifier
Audio Upload
POST /predict
File Save/Check
STT (Whisper)
Text Preprocessing
Classification
JSON Response
UI Rendering
(Alert/Highlight)
Option: Report/Block
Logging/Stats

|데이터 수집처리 정의서


국립국어원
언어정보나눔터
일상대화 말뭉치
일상대화 말뭉치
json파일을 csv로
변환
금융감독원
보이스피싱 실제 사례
음성 데이터
보이스피싱 데이터를
whisper STT 모델을
통해 텍스트로 전환
데이터 불균형 처리를
위해 일반대화 데이터
비복원 추출
일반대화 : 보이스피싱
1:1로 조정

| 프로그램 - 목록

분류	프로그램	기능 명
개발환경	Visual Studio Code	전체 시스템(백엔드 Flask 서버, 모델 실행 코드, Whisper STT, 전처리 스크립트 등) 개발 및 관리에 사용된 주요 통합 개발 환경(IDE).
클라우드/실행 환경	Google Colab Pro	GPU/TPU 기반 모델 학습 및 테스트, 실험 환경 제공
	Naver Cloud Platform	서버 배포, GPU 자원 활용, Flask 백엔드 실행 환경 제공
AI도구/지원 툴	ChatGPT	코드 디버깅, 개발 아이디어 제안, 문서 작성 및 요약 지원
	Gemini	AI 기반 코드 작성 및 아이디어 보조
	Claude	코드 수정, 코드 오류 확인, 프로젝트 문서화 지원
협업도구	Notion	일정 관리, 팀 협업 기록, 회의록 및 문서 공유
프레임워크/라이브러리	Python	모델 학습, 데이터 처리, 백엔드 서버 구현의 기본 언어
	Pytorch	KoBERT 기반 보이스피싱 분류 모델 학습 및 추론을 위한 딥러닝 프레임워크
	Flask	웹 서버 구현 및 백엔드-프론트엔드 연동
	Pandas	데이터 전처리, 가공, 분석을 위한 라이브러리

| 테이블 정의서 - ERD

users

PK	id
	username
	phone
	email
	userid
	password

uploads

PK	id
FK(users.id)	user_id
	filename
	filepath
	upload_time

| 핵심 소스코드(Model)

```
[ ]  # 1. 토크나이저, 모델 로드
     model_name = "skt/kobert-base-v1"
     tokenizer = AutoTokenizer.from_pretrained(model_name)
     model = AutoModelForSequenceClassification.from_pretrained(model_name, num_labels=2)

     device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
     model.to(device)

     # 2. 커스텀 Dataset
     class PhishingDataset(Dataset):
         def __init__(self, texts, labels, tokenizer, max_len=128):
             self.texts = texts
             self.labels = labels
             self.tokenizer = tokenizer
             self.max_len = max_len

         def __len__(self):
             return len(self.texts)

         def __getitem__(self, idx):
             encoding = self.tokenizer(
                 self.texts[idx],
                 truncation=True,
                 padding="max_length",
                 max_length=self.max_len,
                 return_tensors="pt"
             )
             item = {key: val.squeeze(0) for key, val in encoding.items()}
             item["labels"] = torch.tensor(self.labels[idx])
             return item

     # 3. DataLoader 구성
     train_dataset = PhishingDataset(X_train, y_train, tokenizer)
     test_dataset = PhishingDataset(X_test, y_test, tokenizer)

     train_loader = DataLoader(train_dataset, batch_size=16, shuffle=True)
     test_loader = DataLoader(test_dataset, batch_size=16)

     # 4. Optimizer
     optimizer = AdamW(model.parameters(), lr=2e-5)
```

KoBERT 모델과 토크나이저를 로드하고, 커스텀 Dataset과 DataLoader를 통해 입력 텍스트를 학습에 적합한 형태로 변환합니다. 최적화에는 AdamW 옵티마이저를 사용하여 분류 성능을 극대화합니다.

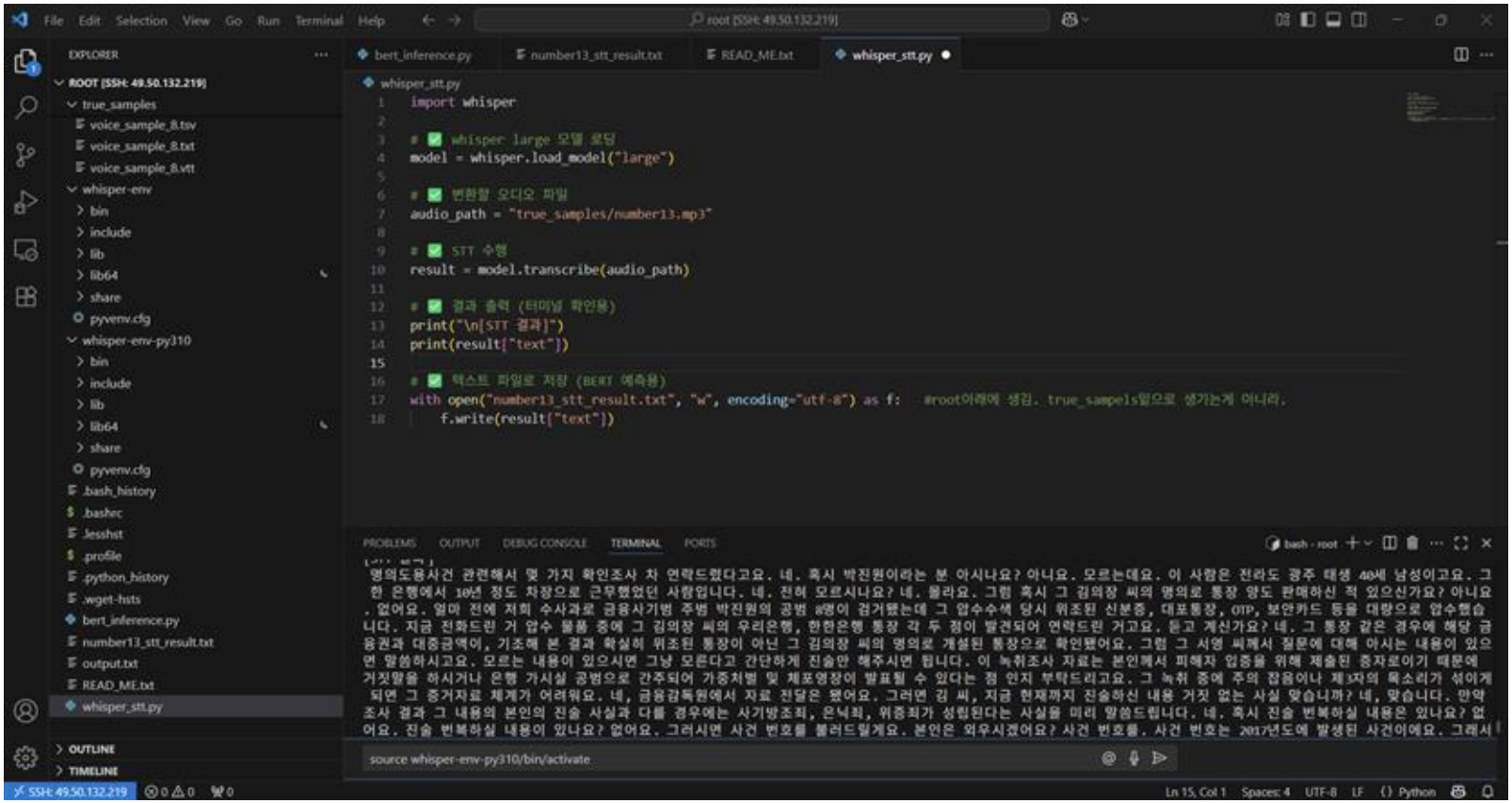
| 핵심 소스코드(Model)

```python
# 5. 학습 루프
epochs = 3
model.train()
for epoch in range(epochs):
    total_loss = 0
    for batch in tqdm(train_loader, desc=f"Epoch {epoch+1}"):
        input_ids = batch["input_ids"].to(device)
        attention_mask = batch["attention_mask"].to(device)
        labels = batch["labels"].to(device)

        outputs = model(input_ids=input_ids, attention_mask=attention_mask, labels=labels)
        loss = outputs.loss

        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
        total_loss += loss.item()

    avg_loss = total_loss / len(train_loader)
    print(f"Epoch {epoch+1} Loss: {avg_loss:.4f}")
```

```
Epoch 1: 100%|██████████| 134/134 [40:57<00:00, 18.34s/it]
Epoch 1 Loss: 0.5313
Epoch 2: 100%|██████████| 134/134 [40:15<00:00, 18.03s/it]
Epoch 2 Loss: 0.3651
Epoch 3: 100%|██████████| 134/134 [40:11<00:00, 17.99s/it]Epoch 3 Loss: 0.3255
```

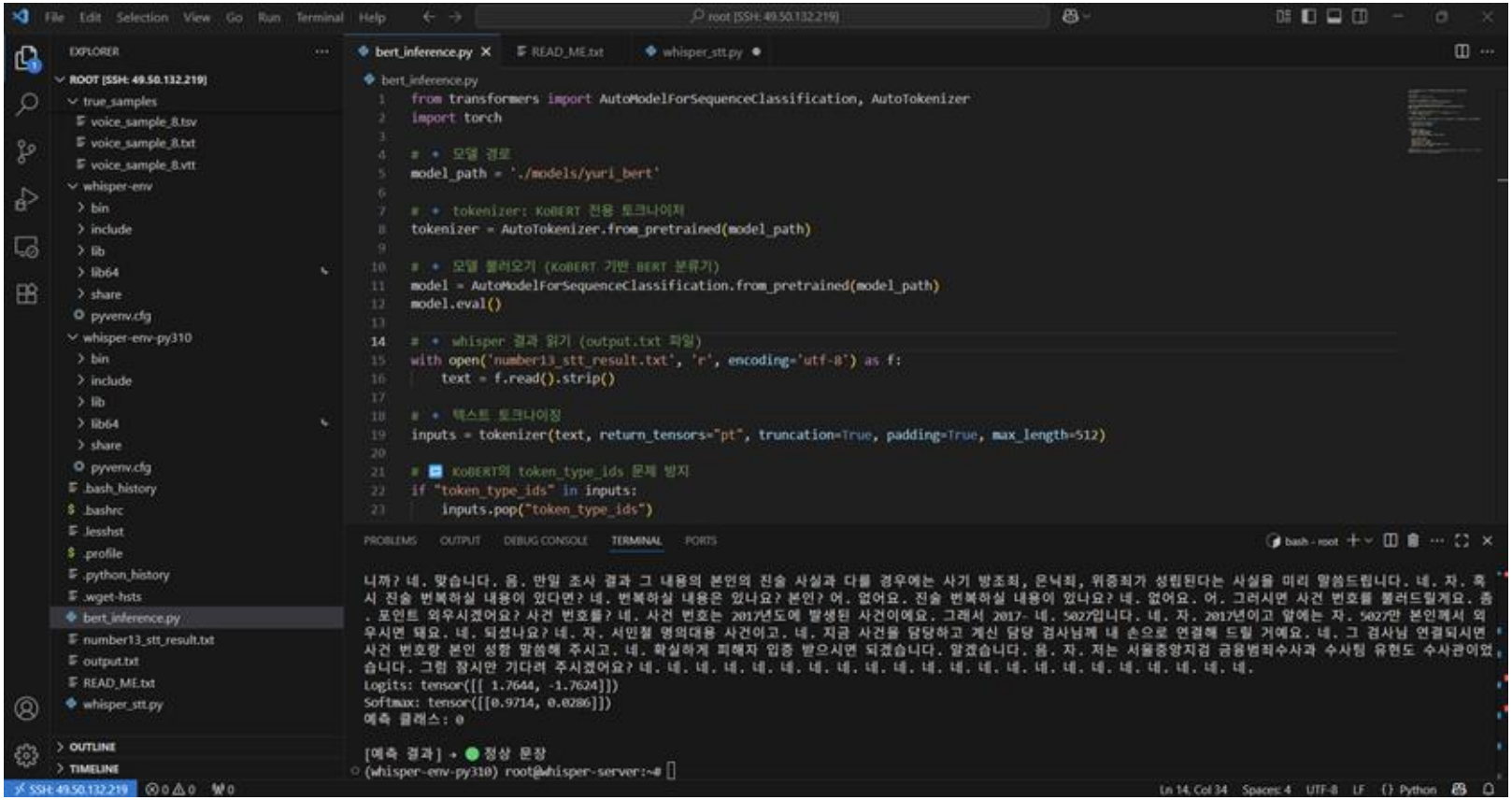
KoBERT 기반 모델을 3에폭 동안 학습하며, 손실 함수 값을 기준으로 역전파와 가중치 갱신을 반복합니다. 그 결과, 손실 값이 0.5313에서 0.3255로 꾸준히 감소하며 모델이 안정적으로 학습되었음을 확인할 수 있습니다.

| 핵심 소스코드(Back-end)



Whisper는 OpenAI가 개발한 고성능 음성 인식(STT, Speech-to-Text) 모델로, 업로드된 음성 파일을 자연어 텍스트로 변환하는 데 사용됩니다.
우리 프로젝트에서는 Whisper의 **large 모델**을 사용하여 다양한 억양과 잡음 환경에서도 높은 정확도의 텍스트 변환을 수행합니다.

| 핵심 소스코드(Back-end)



Whisper로 변환된 텍스트는 KoBERT 기반의 분류 모델에 입력되어, 해당 문장이 보이스피싱인지 여부를 예측합니다.
BERT 모델은 문장 내 의미와 문맥을 파악하여 "정상" 또는 "보이스피싱"으로 분류 결과를 반환합니다.

Thank you